LLM API INTEGRITY AUDITING

Are You Getting What You Pay For?

Auditing Model Substitution in LLM APIs: Why Software Tests Fail and Hardware Attestation Wins

Commercial trust gap: advertised model ≠ served model under cost pressure.

Reference: arxiv.org/abs/2504.04715

Audience: ML researchers • AI security practitioners • cloud infrastructure engineers

The Model Substitution Problem: Economic Incentives Drive Co

Hypothesis framework: H0 = provider serves specified model, H1 = provider serves substitute model.

Why Substitution Is Economically Rational

- API users cannot inspect server-side model weights.
- Inference cost dominates provider margins at scale.
- Cheaper substitutes preserve much of observed quality.
- Audits rely on black-box outputs, enabling plausible deniabili

Four Covert Substitution Types

1) Smaller model: fewer parameters, lower cost.

2) Quantized variant: lower precision (e.g., FP8).

3) Fine-tuned variant: altered behavior, hidden drift.

4) Different family: alternate architecture/provider.

Audit objective: reject H0 only with robust, operationally feasible ev

FP8 Quantization Preserves Most Accuracy — Making Substitution Hard

Original vs FP8 scores from source brief (mean ± CI) across MMLU, GSM8K, MATH, GPQA Diamond.

Model Family	MMLU	GSM8K	MATH	GPQA Diamond
Llama-3-8B: 62.69→62.43	Δ -0.26	61.14→60.90 (Δ -0.24)	20.65→14.91 (Δ -5.74)	22.62→20.14 (Δ -2.48)
Llama-3-70B: 78.05→77.88	Δ -0.17	88.06→87.35 (Δ -0.71)	35.69→35.75 (Δ +0.06)	29.60→33.12 (Δ +3.52)
Gemma-2-9b: 71.86→71.92	Δ +0.06	81.80→79.41 (Δ -2.39)	33.41→32.53 (Δ -0.88)	28.64→27.81 (Δ -0.83)
Qwen2-72B: 82.18→81.98	Δ -0.20	86.72→86.82 (Δ +0.10)	37.39→37.67 (Δ +0.28)	29.93→31.08 (Δ +1.15)
Mistral-7B: 59.15→58.77	Δ -0.38	35.90→32.20 (Δ -3.70)	8.94→7.68 (Δ -1.26)	21.60→22.72 (Δ +1.12)

Interpretation: MMLU/GSM8K shifts are often small relative to uncertainty, weakening black-box detectability.
Even where some task drops exist, mixed results across benchmarks complicate robust forensic attribution.

Bottom line: low-cost FP8 substitution can preserve headline quality while evading software-only audits.

Higher Temperature Amplifies Performance Gaps but Increases Noise

Evidence: figure_1.jpg (benchmark accuracy vs temperature across five models and four tasks).

Observed Pattern

- As temperature rises to 2.0, all models degrade materially.
- Original-vs-substitute gaps can narrow or become inconsistent.
- Increased sampling noise reduces confidence in output-based tests.

Implication

Tuning temperature to expose substitution also injects variance that undermines statistical power and reproducibility.

Log Probability Fingerprinting Fails Due to Inference Nondeterminism

Evidence: figure_5.jpg (same Llama-3-8B, varying stacks: transformers/vLLM + A100/H100).

Failure Mechanism

- Identical model weights can emit different token log-probabilities.
- Sources: software version drift, kernel choices, GPU architecture.
- Production APIs rarely expose full execution metadata for normalization.

Consequence

Log-prob fingerprints are non-stable under realistic deployment variation, so false alarms and misses are unavoidabl

Randomized Routing at Low Rates Makes Statistical Detection Query-Prohib

Attack: serve substitute outputs with probability p, specified model with probability $(1-p)$.

Mixture Distribution

$$P\_mix(y|x) = (1-p)\cdot P\_spec(y|x) + p\cdot P\_sub(y|x)$$

As $p \to 0$, P_mix converges to P_spec.

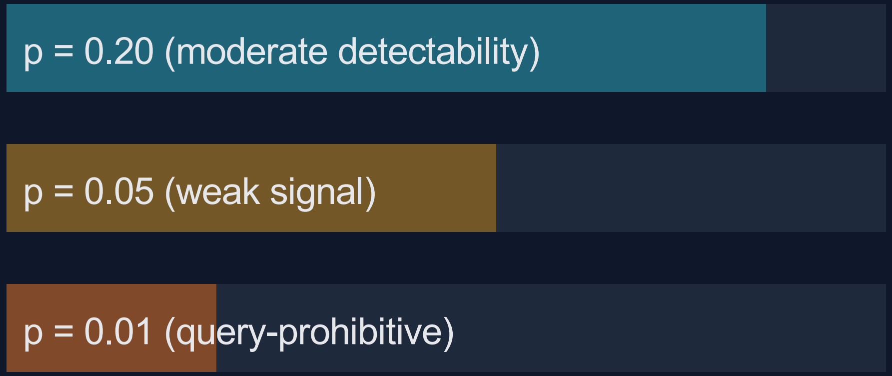
Signal-to-noise ratio shrinks with substitution rate.

Required query budget increases sharply.

Operational effect:

Provider can hide substitution with low-rate routing while preserving economics across aggregate traffic.

Conceptual Detection Difficulty



Low-rate randomized substitution asymptotically defeats black-box

Software-Only Auditing Methods Are Fundamentally Unreliable

Comparison matrix of failure modes observed in realistic API settings.

Method Type	Core Weakness	Practical Limitation
Text-based statistical tests (e.g., MMD on outputs)	Small distribution shifts under subtle substitution, temperature and sampling inject variance.	Needs very large query budgets; weak power against near-equivalent or randomized substitutes.
Log-probability fingerprinting	Same model yields different token log-probs across software stacks and GPU hardware.	Unstable signatures cause false positives/negatives; full execution metadata is often unavailable.
Adversarial countermeasures (routing, benchmark evasion)	Provider can adapt outputs and route traffic to minimize distinguishability from specified model.	Black-box auditors cannot force transparency; attacker controls serving pipeline and observables.

Trusted Execution Environments Provide Provable Cryptographic Model Inte

TEE attestation binds runtime, hardware, and model artifact identity at inference time.

Attested Inference Flow

1) Load signed model weights into TEE enclave

2) Hardware measures runtime + model hash

3) Remote verifier checks signed attestation quote

4) Only verified endpoint is trusted for API calls

Practical path: confidential computing stacks (e.g., NVIDIA-supported TEEs).

Why TEEs Beat Software Audits

- Provable: cryptographic attestation, not inference.
- Efficient: modest overhead vs heavy cryptographic proofs.
- Robust: unaffected by sampling randomness or routing tric
- Deployable: aligns with current cloud hardware roadmaps.

Security pivot

From probabilistic detection of behavior
to verifiable guarantees of execution identity.

Key Takeaways: Close the Verification Gap with Hardware Attesta

1) Model substitution is economically motivated and black-box opaque.

2) FP8 quantization often preserves benchmark accuracy, masking substitution.

3) Inference nondeterminism breaks log-probability fingerprint stability.

4) Randomized low-rate routing makes statistical detection query-prohibitive.

5) TEEs are the only currently actionable path to verifiable LLM API integrity.

Call to action: Standardize attestation interfaces, publish verification evidence, and require integrity guarantees ir